

8

Statement-Level Control Structures

Türkçe Anlatım · Terimler İngilizce

İçerik:

Konu Anlatımı · Review Questions · Problem Set
Programming Exercises

CSE 341 — Programming Languages
Sebesta, *Concepts of Programming Languages*, 12th Ed.

Gebze Technical University · Spring 2026

İçindekiler

1 Giriş: Control Statements’a Genel Bakış	3
1.1 Motivation: Neden Control Statement’lara İhtiyaç Var?	3
1.2 Tarihsel Arka Plan	3
1.3 Tasarım Felsefesi: Writability vs Readability Trade-off	3
1.4 Tüm Selection ve Iteration İçin Ortak Tasarım Sorusu	4
1.5 Control Structure’ların Sınıflandırması	4
2 Selection Statements	4
2.1 Two-Way Selection Statements	4
2.1.1 Tasarım Soruları	5
2.1.2 Control Expression	5
2.1.3 Clause Form	5
2.1.4 Nesting Selectors — Dangling Else Problemi	6
2.1.5 Selector Expression’ları	7
2.1.6 Two-Way Selector Karşılaştırma Tablosu	7
2.2 Multiple-Selection Statements	7
2.2.1 Tasarım Soruları	7
2.2.2 C / C++ / Java / JavaScript: <code>switch</code>	8
2.2.3 C# <code>switch</code> : Reliability İyileştirmesi	9
2.2.4 Ruby <code>case</code>	9
2.2.5 Multiple-Selection Karşılaştırma Tablosu	9
2.2.6 Multiple Selection’ın Implementasyonu	10
2.2.7 Multiple Selection Using <code>if</code> (else-if)	10
2.2.8 Scheme <code>COND</code>	10
3 Iterative Statements	11
3.1 Tasarım Soruları	11
3.2 Counter-Controlled Loops	11
3.2.1 Tasarım Soruları	12
3.2.2 C-Tabanlı Dillerin <code>for</code> Statement’ı	12
3.2.3 Python <code>for</code>	13
3.2.4 Functional Dillerde Counter-Controlled Loop	13
3.2.5 Counter-Controlled Loop Karşılaştırma Tablosu	13
3.3 Logically Controlled Loops	14
3.3.1 Tasarım Soruları	14
3.3.2 C-Tabanlı Dillerde <code>while</code> ve <code>do-while</code>	14
3.4 User-Located Loop Control Mechanisms	15
3.4.1 Tasarım Soruları	15
3.4.2 <code>break</code> , <code>continue</code> , <code>last</code>	15
3.5 Iteration Based on Data Structures	16
3.5.1 C <code>for</code> ile Iterator Simulation	16
3.5.2 Java <code>foreach</code> (Java 5.0+)	16
3.5.3 C# <code>foreach</code>	16
3.5.4 Ruby Blocks ve Iterators	17
3.5.5 Python Generators	17
3.5.6 Iterator Karşılaştırma Tablosu	18

4	Unconditional Branching	18
4.1	goto'nun Gücü ve Tehlikesi	18
4.2	goto Kararları	18
5	Guarded Commands	19
5.1	Guarded Selection: <code>if [] [] fi</code>	19
5.1.1	Semantics: Nondeterminism	19
5.1.2	Klasik Örnek: <code>max(x, y)</code>	20
5.2	Guarded Loop: <code>do [] [] od</code>	20
6	Sonuç ve Değerlendirme	21
7	Bölüm Özeti: Tüm Kavramlar Bir Bakışta	22
8	Sınav İçin Kritik Noktalar	23
9	Review Questions — Soru Çözümleri	23
10	Problem Set — Soru Çözümleri	29
11	Programming Exercises — Ek Çözümler	34

1 Giriş: Control Statements’a Genel Bakış

Bir programın **flow of control**’u (yürütme sırası) farklı seviyelerde incelenebilir. Chapter 7’de *expression*’lar içindeki control flow’u *operator associativity* ve *precedence* kuralları açısından ele almıştık. En üst seviyede ise *program units* (alt programlar, modüller) arasındaki control flow vardır; bu konu Chapter 9 ve 13’ün konusudur. Bu iki uç arasında yer alan **statement’lar arası control flow**, bu bölümün konusudur.

1.1 Motivation: Neden Control Statement’lara İhtiyaç Var?

Imperative dillerde hesaplama, *expression*’ları değerlendirip sonuçları değişkenlere atayarak yapılır. Ancak yalnızca *assignment statement*’lardan oluşan programlar son derece kısıtlıdır. Programları gerçek anlamda esnek ve güçlü kılmak için en az iki temel mekanizmaya daha ihtiyaç vardır:

- Alternatif control flow yolları arasından **seçim** yapma (selection),
- Bir statement veya statement dizisini **tekrar tekrar yürütme** (iteration).

Bu iki yeteneği sağlayan statement’lara **control statement** denir.

Functional dillerde durum farklıdır: hesaplama, *expression*’ları değerlendirme ve fonksiyonları parametrelere uygulama yoluyla yapılır; control flow da yine başka *expression*’lar ve fonksiyonlar tarafından yönetilir. Yine de bunların bir kısmı imperative dillerdeki control statement’lara benzer.

1.2 Tarihsel Arka Plan

İlk başarılı yüksek seviyeli dil olan **Fortran**’ın (1950’lerin ortası) control statement’ları, IBM 704’ün makine diline neredeyse birebir karşılık gelecek şekilde tasarlanmıştı; yani dil tasarımının değil, donanım tasarımının ürünüydüler. O dönemde programlamanın zorlukları henüz iyi anlaşılmadığı için bu statement’lar yeterli görülüyordu.

1965–1975 arası dönemde control statement’lar üzerine yoğun araştırmalar yapıldı. Önemli bir teorik sonuç olarak **Böhm–Jacopini teoremi** (1966) ortaya çıktı: *flowchart ile ifade edilebilen tüm algoritmalar, sadece iki control statement ile (iki yollu seçim ve mantıksal döngü) ifade edilebilir*. Buradan çıkan pratik sonuç, **unconditional branch (goto) statement**’ının teorik olarak gereksiz olduğudur.

Tarihsel Not: Yapılandırılmış Programlama Devrimi

1960’ların sonunda Edsger Dijkstra’nın “*Go To Statement Considered Harmful*” (1968) başlıklı mektubu, programlama dünyasında büyük bir tartışma başlattı. Dijkstra’ya göre goto, programcıyı “programını karmaşıktırmaya davet eden çok ilkel bir araç”tı. Donald Knuth (1974) ise tam yasaklamaya karşı çıktı; bazı durumlarda goto’nun verimlilik kazancının okunabilirlik kaybından daha önemli olabileceğini savundu. Bu tartışma, Pascal (1970) gibi yapılandırılmış dillerin yaygınlaşmasında belirleyici rol oynadı.

1.3 Tasarım Felsefesi: Writability vs Readability Trade-off

Programcılar pratikte teorik sonuçlardan çok **writability** (yazma kolaylığı) ve **readability** (okuma kolaylığı) ile ilgilenir. Yaygın diller, teorik minimum olan iki control statement’tan çok daha fazlasını içerir; çünkü çeşitlilik writability’yi artırır. Örneğin tüm döngüler için yalnızca *logically controlled loop* kullanmak yerine, sayaç tabanlı döngüler için ayrı bir *counter-controlled loop* statement’ı bulunması yazma kolaylığını çok artırır.

Ancak çok fazla statement formu da readability’yi düşürür: kodu okuyan kişi, programcının kullandığı tüm yapıları öğrenmek zorundadır. Aynı şekilde, çok az control statement bulunması da goto gibi düşük seviyeli yapıların kullanımını zorlar; bu da okunabilirliği düşürür.

Tanım: Control Statement ve Control Structure

Control statement: Bir program içinde yürütme akışını yönlendiren statement'tır (selection, iteration, branching).

Control structure: Bir control statement ile onun yürütmesini kontrol ettiği statement koleksiyonunun bütünüdür.

1.4 Tüm Selection ve Iteration İçin Ortak Tasarım Sorusu

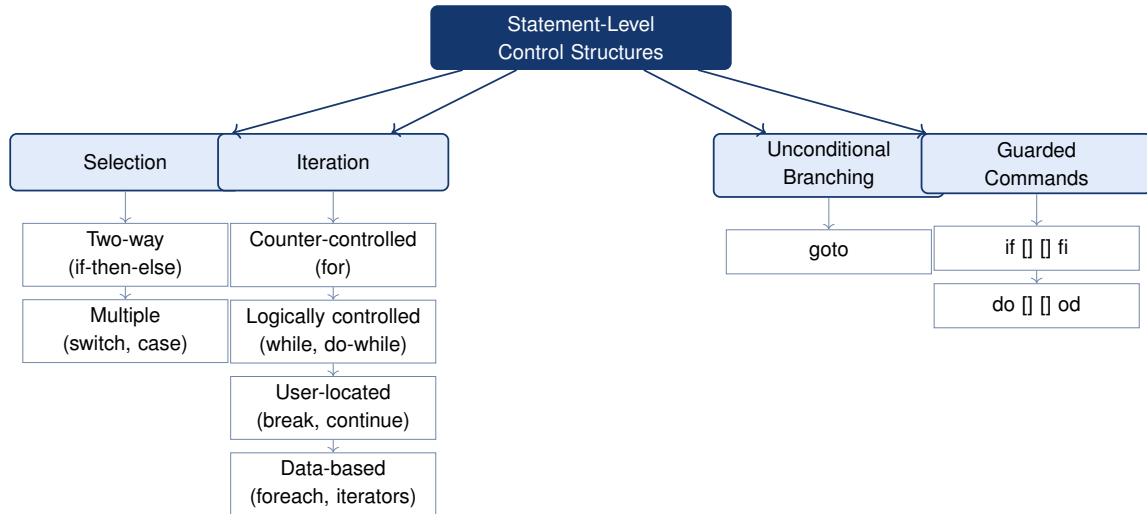
Tüm selection ve iteration yapılarına uygulanan tek bir genel tasarım sorusu vardır: *Control structure çoklu giriş noktasına (multiple entries) sahip olmalı mı?*

Günümüzde **multiple entries**'in esneklik kazandırdığından çok karmaşıklık yarattığı ve readability'yi düşürdüğü kabul edilir. Bu yüzden modern dillerde control structure'lara giriş yalnızca ilk statement'tan yapılır. Multiple entries zaten yalnızca *goto* ve *statement label* içeren dillerde mümkündür.

Sezgisel Açıklama: Neden Multiple Exits Tasarım Sorusu Değil?

Tüm diller bir control structure'dan çıkışın çeşitli formlarına izin verir. Eğer çıkış *yalnızca* structure'ın hemen ardındaki statement'a gidiyorsa (yani normal control flow'un da gideceği yere), readability'ye zarar gelmez. Ancak çıkış programın herhangi bir yerine olabiliyorsa, bu zaten gizli bir *goto*'dur. Yani gerçek mesele **multiple exits** değil, **goto'nun varlığıdır**.

1.5 Control Structure'ların Sınıflandırması



2 Selection Statements

Selection statement, programın iki veya daha fazla yürütme yolu arasından birini seçmesini sağlayan yapıdır. Böhm–Jacopini teoremine göre selection, herhangi bir algoritmayı ifade etmek için zorunlu olan iki temel yapıdan biridir. Selection statement'lar iki ana kategoriye ayrılır: **two-way** (iki yollu) ve **n-way / multiple** (çoklu).

2.1 Two-Way Selection Statements

Çağdaş imperative dillerin two-way selector'ları birbirine çok benzer; ama detaylarda önemli farklar vardır. Genel form:

```
if control_expression
  then clause
  else clause
```

2.1.1 Tasarım Soruları

Two-way selector'lar için üç temel tasarım sorusu vardır:

1. Selection'ı kontrol eden *expression*'ın formu ve tipi nedir?
2. *then* ve *else* clause'ları nasıl belirtilir?
3. İç içe (*nested*) selector'ların anlamı nasıl belirlenmelidir?

2.1.2 Control Expression

Eğer *then* reserved word'ü (veya başka bir sözdizimsel işaretçi) kullanılmıyorsa, kontrol expression'ı parantez içinde belirtilir (C, Java). *then* kullanılan dillerde (Ruby) parantez genellikle gereksizdir.

Tip açısından önemli bir ayrım vardır:

- **C89, Python, C99, C++:** hem aritmetik hem Boolean expression kullanılabilir (sıfır = false, sıfır olmayan = true).
- **Java, C#, Ada:** yalnızca Boolean expression kabul edilir.

Sınav Tuzağı: Aritmetik Kontrol Expression'ı

C'de `if (x = 5) ...` (atama) ile `if (x == 5) ...` (karşılaştırma) farklıdır. Birincisi her zaman `true`'dur (5 sıfır değil) ve sessizce derlenir. Java/C#'ta bu hata derleme zamanında yakalanır, çünkü `x = 5` `int` döner ve Boolean beklenir. Bu fark dil tasarımında **reliability** açısından sıkça karşılaştırılır.

2.1.3 Clause Form

Birçok dilde *then* ve *else* clause'ları tek statement veya **compound statement** (statement bloğu) olabilir. Varyasyonlar:

- **Perl:** clause'lar *zorunlu* olarak compound olmalıdır (tek statement'lık bile olsa).
- **C, C++, Java, C#:** compound statement `{ }` ile yapılır.
- **Python, Ruby:** clause'lar compound statement değil, **statement sequence**'tır; selection'ın sonu reserved word (Ruby'de `end`) veya indentation (Python) ile belirlenir.

Python örneği:

```
if x > y :
    x = y
    print("case 1")
```

Aynı seviyede girintilenmiş tüm statement'lar compound bloğa dahildir. *then* yerine iki nokta (:) kullanılır.

2.1.4 Nesting Selectors — Dangling Else Problemi

Two-way selector'ın dilbilgisi (Chapter 3'te de görüldüğü gibi) şu *ambiguous* formdadır:

```
<if_stmt> -> if <logic_expr> then <stmt>
          | if <logic_expr> then <stmt> else <stmt>
```

Aşağıdaki Java benzeri kodu ele alalım:

```
if (sum == 0)
  if (count == 0)
    result = 0;
else
  result = 1;
```

Soru: else hangi if'e ait? Girinti dıştaki if'e ait gibi gösteriyor; ama (Python ve F# hariç) modern dillerin çoğunda girinti semantik olarak anlamsızdır.

Çözüm yaklaşımları:

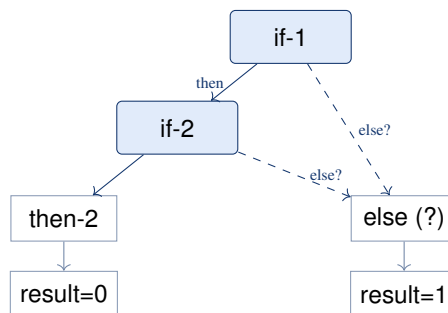
- **Java/C/C++/C#:** Static semantic kuralı: *else daima en yakın eşleşmemiş then ile eşleşir.* Yani örnekte else, içteki if'e bağlanır. Dış if'e bağlamak istersek içtekini compound yapmamız gerekir:

```
if (sum == 0) {
  if (count == 0)
    result = 0;
}
else
  result = 1;
```

- **Perl:** Tüm clause'lar zorunlu compound; problem doğmaz.
- **Ruby:** Selection end ile kapanır:

```
if sum == 0 then
  if count == 0 then
    result = 0
  else
    result = 1
  end
end
```

- **Python, F#:** Indentation semantik anlam taşır.
- **ML:** else-siz if'e izin vermez; problem doğmaz.



Şekil: else'in iki olası bağlanması — Java kuralı içteki if'i seçer.

Sınav Tuzağı: Dangling Else

Sınavda “aşağıdaki kodun çıktısı nedir?” tarzı sorularda Java/C kuralı **en yakın eşleşmemiş then**’dir; girinti yanıltıcıdır. Aynı kodu Python’a çevirdiğinizde girinti gerçekten anlam taşır; bu nedenle anlam **değişebilir**. Diller arası karşılaştırma sorularında bu farkı mutlaka belirtin.

2.1.5 Selector Expression’ları

Functional dillerde (**ML, F#, Lisp**) selector bir statement değil, **expression**’dır; değer üretir ve başka her expression’ın geçebileceği yerde geçebilir. F# örneği:

```
let y =
  if x > 0 then x
  else 2 * x;;
```

F#’ta then clause’unun döndürdüğü değer tipi else clause’unun tipiyle aynı olmalıdır. else yoksa then ancak unit tip (()) döndürebilir; bu “değer yok” anlamındadır.

Neden Önemli? Expression vs Statement Ayrımı

If-as-statement: yan etki için kullanılır (atama, I/O). *If-as-expression*: değer üretir, atanabilir, parametre olabilir, başka expression’ın içinde geçebilir. Functional dilde her şey expression olduğundan, kontrol akışı da değer üretmek zorundadır. Bu, *ternary operator* (?) ile imperative dillerde kısmen taklit edilir.

2.1.6 Two-Way Selector Karşılaştırma Tablosu

Dil	Kontrol Expr	Nesting Çözümü
C, C++	arith / Boolean	nearest-then kuralı; compound { }
Java, C#	yalnızca Boolean	nearest-then kuralı; compound { }
Perl	arith / Boolean	zorunlu compound (problem yok)
Python	arith / Boolean	indentation (problem yok)
Ruby	Boolean (truthy)	end reserved word
F#, ML	Boolean	else-siz if yasak (ML); expression
Ada	Boolean	end if reserved word

2.2 Multiple-Selection Statements

Multiple-selection statement, herhangi sayıda statement veya statement grubundan birinin seçilmesini sağlar. Two-way selector’ın genelleştirilmiş halidir; iki yollu seçim multiple selector ile yapılabilir, tersi de mümkündür ama hantaldır.

İkiden fazla yol arasından seçim yapmak çok yaygın bir ihtiyaçtır. Bunu nested two-way selector ve goto ile inşa etmek mümkün olsa da sonuç *cumbersome, unreliable* ve okuması zor olur.

2.2.1 Tasarım Soruları

1. Seçimi kontrol eden expression’ın formu ve tipi nedir?
2. Seçilebilir segment’ler nasıl belirtilir?
3. Bir yürütmede yalnızca tek bir segment mi yürütülür?

4. Case değerleri nasıl belirtilir?
5. Seçici expression hiçbir case'e uymazsa ne olur?

2.2.2 C / C++ / Java / JavaScript: switch

Genel form:

```
switch (expression) {  
    case constant_expression_1: statement_1;  
    ...  
    case constant_n: statement_n;  
    [default: statement_n_plus_1]  
}
```

- Kontrol expression'ı ve case sabitleri **discrete** (integer, char, enum) olmalıdır.
- Seçilebilir kısımlar tek statement, sequence veya block olabilir.
- default opsiyoneldir; uymayan değerler için.
- **Implicit branch yoktur** (fall-through): break konmazsa kontrol bir sonraki case'e "akar".

Fall-through örneği (hatalı — her seferinde error mesajı basar):

```
switch (index) {  
    case 1:  
    case 3: odd += 1; sumodd += index;  
    case 2:  
    case 4: even += 1; sumeven += index;  
    default: printf("Error in switch, index = %d\n", index);  
}
```

break ile düzeltilmiş versiyon:

```
switch (index) {  
    case 1:  
    case 3: odd += 1;  
            sumodd += index;  
            break;  
    case 2:  
    case 4: even += 1;  
            sumeven += index;  
            break;  
    default: printf("Error in switch, index = %d\n", index);  
}
```

Sınav Tuzağı: Fall-Through Reliability'yi Düşürür

C'nin switch tasarımı **flexibility** (kasıtlı fall-through) için **reliability**'yi feda eder. break unutulursa derleyici uyarır; mantık sessizce bozulur. ALGOL 68'den ilham alır. Çalışmalar fall-through'un nadiren kasten kullanıldığını göstermiştir.

2.2.3 C# switch: Reliability iyileştirmesi

C# iki önemli farkla gelir:

1. Her segment **açık bir unconditional branch** ile bitmek zorundadır (break veya goto case). Implicit fall-through derleme hatasıdır.
2. Kontrol expression'ı ve case değerleri **string** olabilir.

```
switch (value) {
    case -1:
        Negatives++;
        break;
    case 0:
        Zeros++;
        goto case 1;    // explicit fall-through
    case 1:
        Positives++;
        break;
    default:
        Console.WriteLine("Error in switch \n");
        break;
}
```

2.2.4 Ruby case

Ruby'de iki form vardır; ikisi de **expression**'dır (en son değerlendirilen ifadenin değerini döner). Boolean tabanlı form, nested if listesine semantik olarak yakındır:

```
case
  when Boolean_expression then expression
  ...
  when Boolean_expression then expression
  [else expression]
end
```

```
leap = case
  when year % 400 == 0 then true
  when year % 100 == 0 then false
  else year % 4 == 0
end
```

2.2.5 Multiple-Selection Karşılaştırma Tablosu

Dil	Case Tipi	Fall-through	Default Davranış
C, C++	integer, char, enum	implicit (kasıtlı)	yoksa hiçbir şey
Java	integer, enum, String*	implicit	yoksa hiçbir şey
JavaScript	herhangi	implicit	yoksa hiçbir şey
C#	discrete, string	yasak; explicit goto case	zorunlu sonlandırma
PHP	string, int, double	implicit	yoksa hiçbir şey
Ruby (case)	herhangi (Boolean form)	yok	else clause
Python	— (yok; elif kullanılır)	—	—

*Java 7+ sürümünde String.

2.2.6 Multiple Selection'ın Implementasyonu

Multiple-selection statement aslında **n-yollu bir branch**'tir. Üç yaygın implementation yaklaşımı vardır:

Yöntem	Ne Zaman Uygun?	Not
Linear conditional branches	$n < 10$	Ortalama $n/2$ test
Hash table	$n \geq 10$, ayırık değerler	Yaklaşık sabit zaman
Binary search table	$n \geq 10$, range'li case'ler (Ruby)	$\log n$
Jump table (array)	Range küçük, yoğun	Index ile $O(1)$

2.2.7 Multiple Selection Using if (else-if)

switch bazı durumlar için yetersizdir: özellikle seçim ordinal tip yerine **Boolean expression** üzerinden yapılacaksa. Bu durumda nested two-way selector kullanılır; ancak readability için Python ve Perl gibi diller **else-if** kısaltması (Python'da `elif`) sunar.

```
if count < 10 :
    bag1 = True
elif count < 100 :
    bag2 = True
elif count < 1000 :
    bag3 = True
else :
    bag4 = True
```

else-if'in operational semantics (E'ler logic expr, S'ler statement):

```
if E1 goto 1
if E2 goto 2
...
1: S1
   goto out
2: S2
   goto out
out: ...
```

Önemli ayrım: **switch**'te tüm E'ler **tek bir expression**'ın farklı sabitlerle karşılaştırması iken, else-if zincirinde E'ler **birbirinden bağımsız** Boolean expression'lardır. Bu yüzden else-if multiple-selection'ın *daha genel* formudur.

2.2.8 Scheme COND

Scheme'de multiple selector, COND özel formudur:

```
(COND
  (predicate_1 expression_1)
  (predicate_2 expression_2)
  ...
  (predicate_n expression_n)
  [(ELSE expression_n_plus_1)]
)
```

Predicate'ler sırayla değerlendirilir; #T dönen ilk predicate'in expression'ı değerlendirilir ve sonuç COND'un değeri olur.

```
(COND
  ((> x y) "x is greater than y")
  ((< x y) "y is greater than x")
  (ELSE    "x and y are equal")
)
```

Mini Sonuç: Selection Statement'lar

Two-way selector'ın temel tasarım sorunu **dangling else**'tir; çözümler: nearest-then kuralı, zorunlu compound, terminator reserved word, indentation. Multiple selector'ın tasarımı **flexibility ile reliability** arasında bir trade-off'tur: C fall-through'a izin verir (reliability düşük), C# zorunlu sonlandırma ister (reliability yüksek). Else-if zinciri Boolean expression'lar arasında seçim yapar, switch sabit karşılaştırması yapar.

3 Iterative Statements

Iterative statement (yaygın adıyla **loop**), bir statement veya statement koleksiyonunun sıfır, bir veya daha fazla kez yürütülmesini sağlayan yapıdır. Plankalkül'den günümüze kadar her programlama dili bir iteration mekanizması içermiştir.

Neden Önemli?

Iteration olmadan programcı her eylemi tek tek yazmak zorunda kalır; programlar devasa, esnek olmayan ve pratikte yazılması imkânsız hale gelirdi. Iteration, bilgisayarın gücünün özüdür.

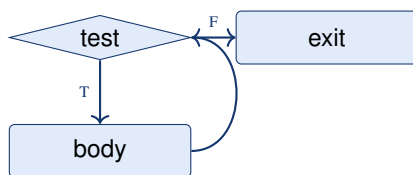
3.1 Tasarım Soruları

İki temel tasarım sorusu loop tasarımını şekillendirir:

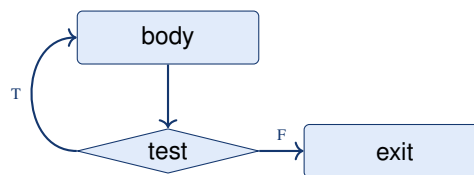
1. **Iteration nasıl kontrol edilir?** (logical / counting / kombine)
2. **Kontrol mekanizması loop'un neresinde olur?** (top / bottom / user-located)

“Top” ve “bottom” fiziksel değil mantıksal anlamdadır: **pretest**'te kontrol body'den önce, **posttest**'te body'den sonra değerlendirilir.

Pretest (while)



Posttest (do-while)



3.2 Counter-Controlled Loops

Bir **counting iterative control statement**, sayı değerini tutan bir **loop variable** (döngü değişkeni) içerir. Ayrıca **initial value**, **terminal value** ve ardışık değerler arasındaki fark olan **stepsize** belirtilir. Bu üçüne birlikte **loop parameters** denir.

Tanım: Loop Variable, Loop Parameters, Stepsize

Loop variable: Her iterasyonda güncellenen, count tutan değişken.

Loop parameters: initial value, terminal value, stepsize üçlüsü.

Stepsize: Loop variable'ın ardışık iki değeri arasındaki fark.

3.2.1 Tasarım Soruları

1. Loop variable ve parameters'ın **tipi** ve **scope**'u ne olmalıdır?
2. Loop body içinde loop variable veya parameters **değiştirilebilir mi**? Değiştirilirse loop'u etkiler mi?
3. Loop parameters **ne zaman** (her iterasyonda mı, yalnız bir kez mi) değerlendirilir?
4. Loop bittikten sonra loop variable'ın **değeri** nedir?

Fortran 90 loop sonrasında loop variable'ı undefined yapar; Ada loop variable'ın scope'unu loop'un kendisiyle sınırlar (dışarıda zaten yoktur).

3.2.2 C-Tabanlı Dillerin for Statement'ı

Genel form:

```
for (expression_1; expression_2; expression_3)
    loop body
```

- expression_1: **init**, sadece bir kez yürütülür.
- expression_2: **loop control**, her iterasyondan önce değerlendirilir; sıfır/false ise döngü biter.
- expression_3: her iterasyondan sonra yürütülür (genellikle artırma).

Operational semantics:

```
    expression_1
loop:  if expression_2 = 0 goto out
      [loop body]
      expression_3
      goto loop
out:   ...
```

Esnek for örneği (çoklu loop variable):

```
for (count1 = 0, count2 = 1.0;
     count1 <= 10 && count2 <= 100.0;
     sum = ++count1 + count2, count2 *= 2.5)
; /* empty body: all work in for header */
```

C99/C++/Java/C# farkları:

- Init kısmında değişken *declaration* yapılabilir; scope loop body'sidir.
- Java/C# için kontrol expression'ı yalnızca boolean olabilir.

```
for (int count = 0; count < len; count++) {
    /* count's scope ends here */
}
```

3.2.3 Python for

Genel form:

```
for loop_variable in object:
    # loop body
[else:
    # else clause]
```

- loop_variable, her iterasyonda object'ten bir değer alır.
- Body içinde loop variable değiştirilebilir; ama **loop davranışını etkilemez** (sonraki iterasyonda yine sıradaki değer atanır).
- else clause'u, döngü **normal şekilde sona ererse** (yani break ile değil) çalışır.

range fonksiyonu:

```
range(5)          # [0, 1, 2, 3, 4]
range(2, 7)       # [2, 3, 4, 5, 6]
range(0, 8, 2)    # [0, 2, 4, 6]
```

range **üst sınırı asla döndürmez**.

Sınav Tuzağı: Python for Davranışları

(1) range(0, 8, 2) sondaki 8'i içermez; (2) Python'da for'un else'i sadece break olmadan biten döngülerde çalışır — bu çoğu öğrencinin tuzağa düştüğü noktadır; (3) Loop variable'ı body içinde değiştirmek loop akışını değiştirmez.

3.2.4 Functional Dillerde Counter-Controlled Loop

Saf functional dillerde counter variable yoktur; iteration yerine **recursion** kullanılır. F# örneği:

```
let rec forLoop loopBody reps =
    if reps <= 0 then
        ()
    else
        loopBody()
        forLoop loopBody (reps - 1);;
```

rec reserved word'ü fonksiyonun rekürsif olduğunu gösterir. () (empty parentheses) F# unit tipindedir; F#'ta else her zaman zorunlu olduğundan “hiçbir şey yapma” anlamı vermek için kullanılır.

3.2.5 Counter-Controlled Loop Karşılaştırma Tablosu

Dil	Forma	Param. mi?	değiştirilebilir	Loop var. sonrası
C, C++	üç expr.; çok esnek	evet		son değer
Java, C#	boolean kontrol	evet		son değer
Python	for x in iter	evet (etkisiz)		son değer
Fortran 90	DO i = a,b,s	hayır		undefined
Ada	for I in a..b	hayır		yalnız loop scope'u
F#	recursion	—		—

3.3 Logically Controlled Loops

Bazen tekrar sayısını sayaç değil, **Boolean expression** belirler. Bu durumda **logically controlled loop** kullanılır. Bu loop'lar counter-controlled loop'lardan *daha geneldir*: her counting loop mantıksal loop ile yazılabilir; tersi her zaman doğru değildir.

3.3.1 Tasarım Soruları

1. Pretest mi posttest mi olmalı?
2. Counting loop'un özel formu mu, ayrı statement mı?

3.3.2 C-Tabanlı Dillerde while ve do-while

```
while (control_expression)
    loop body
```

```
do
    loop body
while (control_expression);
```

C# örneği:

```
sum = 0;
indat = Int32.Parse(Console.ReadLine());
while (indat >= 0) {
    sum += indat;
    indat = Int32.Parse(Console.ReadLine());
}

value = Int32.Parse(Console.ReadLine());
do {
    value /= 10;
    digits++;
} while (value > 0);
```

Operational semantics:

<pre>while: loop: if !control goto out [body] goto loop out: ...</pre>	<pre>do-while: loop: [body] if control goto loop</pre>
--	--

Sınav Tuzağı: Posttest Loop Body En Az Bir Kez Çalışır

do-while body'si **her zaman en az bir kez** yürütülür. Programcılar bunu unuttuğunda, başlangıç koşulu *önceden* sağlanmıyorsa hatalı sonuç doğar. Bu yüzden posttest loop'lar nispeten nadirdir; modern dillerin çoğu pretest tercih eder.

F#'ta pretest while simulation:

```
let rec whileLoop test body =
    if test() then
        body()
        whileLoop test body
```

```
else
    ();;
```

3.4 User-Located Loop Control Mechanisms

Bazen programcı, loop kontrolünü loop'un başı veya sonu yerine **kendi seçtiği bir yerde** yapmak ister. Bu durum için modern diller **user-located exit** mekanizması sunar.

3.4.1 Tasarım Soruları

1. Exit koşullu mu olmalı, exit'in kendisi mi koşulu içermeli?
2. Yalnız tek bir loop'tan mı çıkılır, dış loop'lara da erişilebilir mi?

3.4.2 break, continue, last

- **C, C++, Python, Ruby, C#:** break koşulsuz, etiketsiz.
- **Java:** break label; (etiketli, dış loop'a çıkar).
- **Perl:** last LABEL; (etiketli).
- **C, C++, Python:** continue, mevcut iterasyonun geri kalanını atlar; loop'u terminate *etmez*.

Java'da etiketli break ile iç içe loop'tan çıkış:

```
outerLoop:
    for (row = 0; row < numRows; row++)
        for (col = 0; col < numCols; col++) {
            sum += mat[row][col];
            if (sum > 1000.0)
                break outerLoop;
        }
```

continue ile break farkı:

```
/* continue: skip rest of iteration */
while (sum < 1000) {
    getNext(value);
    if (value < 0) continue;    /* go back to test */
    sum += value;
}

/* break: terminate the loop */
while (sum < 1000) {
    getNext(value);
    if (value < 0) break;      /* exit loop */
    sum += value;
}
```

Sezgisel Açıklama: User-Located Exit “Disiplinli goto”dur

break, continue, last aslında *kamuflajlanmış goto*'lardır. Ama hedefleri ya loop'tan çıkış noktasıdır ya da loop'un başıdır; her yere atlamazlar. Bu sebeple readability'ye zarar vermezler, hatta düzeltirler.

Dil	Exit Mekanizması	Etiketli mi?
C, C++	break, continue	hayır
Java	break label, continue label	evet
C#	break, continue, goto	goto ile dolaylı
Python	break, continue	hayır
Ruby	break, next, redo	hayır
Perl	last LABEL, next LABEL	evet

3.5 Iteration Based on Data Structures

Data-based iteration, kullanıcı tanımlı bir veri yapısını, kullanıcı tanımlı bir **iterator** fonksiyonu ile dolaşmaya yarar. Iterator her çağrıldığında yapıdan bir sonraki elemanı döner; tüm elemanlar ziyaret edildikten sonra sonlanma sinyali verir.

Tanım: Iterator

Iterator, bir veri yapısının elemanlarını sırayla, her birini bir kez olmak üzere döndüren **history-sensitive** fonksiyondur. “History- sensitive” olması, hangi elemanın daha önce sunulduğunu hatırlaması demektir.

3.5.1 C for ile Iterator Simulation

```
for (ptr = root; ptr == null; ptr = traverse(ptr)) {
    /* process *ptr */
}
```

Burada traverse bir önceki düğümden bir sonrakine geçen kullanıcı fonksiyonudur.

3.5.2 Java foreach (Java 5.0+)

```
for (String myElement : myList) {
    /* process myElement */
}
```

myList, Iterable interface’ini implemente eden herhangi bir koleksiyon olabilir.

3.5.3 C# foreach

```
List<String> names = new List<String>();
names.Add("Bob");
names.Add("Carol");
names.Add("Alice");

foreach (String name in names)
    Console.WriteLine(name);
```

Kullanıcı tanımlı koleksiyonlar IEnumerator interface’ini implemente ederek foreach’le uyumlu hale gelir.

3.5.4 Ruby Blocks ve Iterators

Ruby’de **block**, { } veya do .. end ile sınırlandırılmış kod dizisidir. Block, bir method’a parametre olarak gönderilebilir; method bu block’u `yield` ile çağırır.

```
>> 4.times {puts "Hey!"}
Hey!
Hey!
Hey!
Hey!
=> 4
```

Block parametreleri | ... | içinde:

```
>> list = [2, 4, 6, 8]
>> list.each {|value| puts value}
2
4
6
8
=> [2, 4, 6, 8]
```

`upto` ile counting:

```
1.upto(5) {|x| print x, " "}
# 1 2 3 4 5
```

Ruby’nin `for` statement’ı yoktur; `for x in 1..5` ifadesi derleyici tarafından `upto` çağrısına dönüştürülür.

3.5.5 Python Generators

Python’da `yield` statement’ı içeren method’a **generator** denir. Sıradan bir method gibi yazılır ama her çağrıldığında bir önceki `yield`’in bittiği yerden devam eder.

```
class MyStructure:
    # constructor and other methods...
    def traverse(self):
        # if there is another node:
        #   set nod to next node
        # else:
        #   return
        yield nod
```

Neden Önemli? Generators ve Lazy Evaluation

Generator yaklaşımı, tüm elemanları belleğe almak yerine ihtiyaç oldukça üretir. Çok büyük veri yapılarında bu **lazy evaluation** bellek tasarrufu sağlar ve sonsuz dizilerle bile çalışmaya izin verir.

3.5.6 Iterator Karşılaştırma Tablosu

Dil	Mekanizma	Tip
C, C++	for + traversal func	manuel
Java	for-each, Iterable	predefined & user
C#	foreach, IEnumerator	predefined & user
Python	for in, generator (yield)	predefined & user
Ruby	block + each, times, upto, yield	built-in & user
F#	sequence expressions, seq	predefined & user

4 Unconditional Branching

Unconditional branch (yani `goto`) statement'ı, yürütme kontrolünü programın belirtilen bir noktasına aktarır. 1960'ların sonundaki en hararetli dil tasarımı tartışması, bu statement'ın yüksek seviyeli dillerde bulunup bulunmaması üzerineydi.

4.1 `goto`'nun Gücü ve Tehlikesi

`goto` en güçlü control statement'tır: tüm diğer control yapıları `goto` ve seçici ile inşa edilebilir. Ama bu güç aynı zamanda en büyük tehlikedir. Sınırsız `goto` kullanımı:

- Programları okunamaz hale getirir.
- Doğru yürütme sırasını anlamayı zorlaştırır.
- Bakım maliyetini astronomik olarak artırır.

Sebeptir: `goto`, **metinsel sıraya bakmaksızın** herhangi bir statement'ı başka bir statement'ın ardına gelmeye zorlar. Oysa okuma, en iyi yukarıdan aşağıya akarken işler.

Tarihsel Tartışma: "GOTO Considered Harmful"

Dijkstra (1968) mektubunda `goto`'yu "*programını karmaşıktırmaya çağrı niteliğinde, çok ilkel bir yapı*" olarak nitelendirdi. Knuth (1974) tam yasaklamaya karşı çıktı; bazı optimizasyon durumlarında `goto`'nun verimliliğin *readability*'den daha önemli olabileceğini savundu. Sonuçta dengeli bir görüş kazandı: *goto kalsın, ama disiplinle kullanılsın*.

4.2 `goto` Kararları

Dil	<code>goto</code> Var mı?
C, C++	evet (her yere)
C#	evet (özellikle <code>switch</code> içinde <code>goto case</code>)
Java	yok (yerine etiketli <code>break/continue</code>)
Python	yok
Ruby	yok
Ada	evet, kısıtlı

Sınav Tuzağı: Loop Exit'ler de Birer goto'dur

`break`, `continue`, `last` aslında **disiplinli goto**'lardır. Java `goto`'yu yasaklamış olsa da etiketli `break` kalmıştır. Yani `goto`'nun *tam yasaklanması* mümkün değildir; mesele hangi formda ve nereye atlamasına izin verildiğidir.

5 Guarded Commands

Guarded command'lar, Dijkstra (1975) tarafından önerilen farklı bir selection ve loop yapı türüdür. Temel motivasyonu, programın **correctness**'ini test sırasında değil *geliştirme sırasında* garanti edebilen bir programlama metodolojisini desteklemektir.

Neden Önemli? Formal Verification ve Concurrency

Guarded command'lar bir dilin ana control statement'ı olarak yaygınlaşmadı, ama iki kritik etkisi oldu: (1) **CSP** (Hoare, 1978) ve dolayısıyla modern concurrency mekanizmalarının (Ada, occam) temelini oluşturdular. (2) **Haskell**'de fonksiyon tanımlarının syntax'ı guarded command'lardan ilham alır.

5.1 Guarded Selection: `if [] [] fi`

Genel form:

```
if <Boolean expr> -> <statement>
[] <Boolean expr> -> <statement>
[] ...
[] <Boolean expr> -> <statement>
fi
```

Kapanış reserved word'ü `fi`, açılış `if`'in tersten yazımıdır (ALGOL 68'den). `[]` işaretleri “fatbar” adıyla bilinir; her *guard* (Boolean expr) ve karşılık gelen statement birlikte bir *guarded clause* oluşturur.

5.1.1 Semantics: Nondeterminism

- Statement'a her gelindiğinde **tüm guard'lar değerlendirilir**.
- Birden fazlası true ise, içlerinden **biri nondeterministic** olarak seçilir.
- Hiçbiri true değilse **run-time error** oluşur.

Bu son nokta tasarımın özüdür: programcı tüm olasılıkları *açıkça* listelemeye zorlanır. Sessizce “hiçbir şey yapmamak” yok.

Örnek:

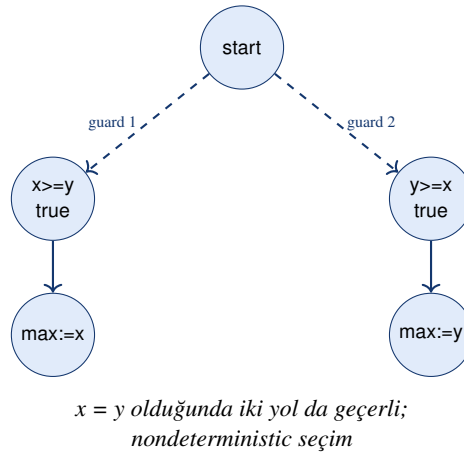
```
if i = 0 -> sum := sum + i
[] i > j -> sum := sum + j
[] j > i -> sum := sum + k
fi
```

$i = 0$ ve $j > i$ ise, birinci ile üçüncü clause arasında nondeterministic seçim yapılır. $i = j$ ve $i \neq 0$ ise hiçbir guard true olmaz; run-time error.

5.1.2 Klasik Örnek: max(x, y)

```
if x >= y -> max := x
[] y >= x -> max := y
fi
```

$x = y$ olduğunda iki clause da true'dur; hangisi seçilirse seçilsin sonuç aynıdır. Bu **abstraction**'ın özünü budur: “hangisinin seçildiği önemli değil” bilgisi nondeterminism ile ifade edilir, programcı yapay bir tercih yapmaya zorlanmaz.



5.2 Guarded Loop: do [] [] od

```
do <Boolean expr> -> <statement>
[] <Boolean expr> -> <statement>
[] ...
[] <Boolean expr> -> <statement>
od
```

Semantics:

- Her iterasyonda tüm guard'lar değerlendirilir.
- Birden fazla true ise nondeterministic seçim yapılır; statement yürütülür ve guard'lar yeniden değerlendirilir.
- Tüm guard'lar aynı anda false ise loop sonlanır.

Klasik örnek (4 değişkenin sıralanması): $q_1 \leq q_2 \leq q_3 \leq q_4$ olacak şekilde sırala:

```
do q1 > q2 -> temp := q1; q1 := q2; q2 := temp;
[] q2 > q3 -> temp := q2; q2 := q3; q3 := temp;
[] q3 > q4 -> temp := q3; q3 := q4; q4 := temp;
od
```

Bu kod aslında *bubble sort*'un nondeterministic versiyonudur. Her adımda hangi out-of-order çiftin düzeltilileceği belirsizdir, ama her olası seçim sonunda doğru sonuca götürür.

Sınav Tuzağı: Nondeterminism Random Demek Değildir

Nondeterministic seçimde implementation **tutarlı** bir kural kullanabilir (örneğin her zaman ilk true guard'ı seçmek). Önemli olan, **programın correctness'ının hangi seçim yapıldığına bağlı olmaması**dır. Bu, programcının açıkça belirtmek zorunda olduğu bir özelliktir.

6 Sonuç ve Değerlendirme

Teorik minimum: Sequence + selection + pretest logical loop (Böhm–Jacopini). Pratikte bu minimum yeterli değildir; writability ve readability için daha çok yapı eklenir.

Pratik gerçek: Çoğu modern dil benzer control yapıları içerir, ancak detayda farklılıklar (goto var mı, switch reliable mı, else-if kısaltması var mı) sürer. Wirth ve Hoare basitliği savunur; gerçek dünyada her dil bu basitlik ile zenginlik arasında bir denge tutturur.

goto tartışması: Sona ermiş değildir; C# goto içerirken Java içermez. Ortak görüş: goto kalsın, ama disiplinle kullanılsın.

Guarded command’lar ana akıma giremedi, ama formal verification ve concurrency için temel oluşturdular.

7 Bölüm Özeti: Tüm Kavramlar Bir Bakışta

Kavram	Tanım	Kritik Nokta	İlgili Diller
Control statement	Yürütme akışını yönlendiren statement	Selection, iteration, branching	Tüm imperative diller
Control structure	Control statement + kontrol ettiği body	Multiple-entry kötü, multiple-exit OK	Hepsi
Two-way selection	if-then-else	Dangling else problemi	C, Java, Python, Ruby
Dangling else	İki then'e tek else	Çözüm: kural / compound / terminator	C, Java (kural); Perl, Ruby
Selector expression	Değer üreten if	ML, F# tip uyumu zorunlu	ML, F#, Lisp
Multiple selection	switch / case	Fall-through reliability düşürür	C, C#, Ruby, Scheme
Fall-through	Implicit branch yokluğu	C: var; C#: yasak	C, ALGOL 68
else-if (elif)	Boolean expr listesinde seçim	switch'ten daha geneldir	Python, Perl
Counter-controlled loop	Sayaçla iteration	Loop var. scope'u tartışmalı	Hepsi
Loop parameters	init, terminal, stepsize	Bir kez mi, her seferde mi değerlendirilir	Fortran 90, Ada, C
Pretest loop	Kontrol body'den önce	0 kez yürütülebilir	while
Posttest loop	Kontrol body'den sonra	En az 1 kez yürütülür	do-while
Logically controlled loop	Boolean ile tekrar	Counting'den daha genel	C, Java
User-located exit	break, continue, last	Disiplinli goto	C, Java, Perl
Labeled break	Dış loop'a çıkış	Java, Perl	Java, Perl
Iterator	History-sensitive elem. üretici	foreach ile birlikte	Java, C#, Ruby
Generator	yield ile lazy iterator	Python, JS	Python, JS
Block (Ruby)	Anonim method-paramı	yield ile çağrılır	Ruby
goto	Unconditional branch	Tehlikeli; disiplinli kullan	C, C#; yok: Java, Python
Guarded command	Dijkstra'nın nondet. seçimi	Hepsi false → runtime error	CSP, Haskell
Nondeterminism	Birden fazla geçerli yol	Implementation seçer	Guarded commands, CSP
COND (Scheme)	Multiple selector special form	Predicate #T dönen ilk seçilir	Scheme, Lisp

8 Sınav İçin Kritik Noktalar

1. **Böhm–Jacopini teoremi:** Sequence + selection + pretest loop her algoritmayı ifade etmeye yeter. Bu yüzden goto teorik olarak gereksizdir.
2. **Dangling else:** Java/C/C++/C# kuralı: *else daima en yakın eşleşmemiş then ile eşleşir.* Girinti yanıltıcıdır.
3. **C89’da Boolean tip yok:** aritmetik kontrol expression’ı kullanılır, sıfır = false. Java/C# bunu yasaklar; `if (x = 5)` Java’da derleme hatası, C’de sessiz tehlike.
4. **C switch fall-through** reliability’yi düşürür; C# implicit fall-through’u yasaklayarak bu hatayı kapatır.
5. **else-if zinciri $\neq$ switch:** else-if Boolean expression’lar arasında seçim yapar; switch tek expression’ın sabitlerle karşılaştırmasıdır.
6. **Python for’un else’i:** sadece döngü break’siz bittiğinde çalışır. Sık karıştırılan tuzak.
7. **Python range:** üst sınırı asla içermez. `range(5)  $\rightarrow$  0..4.`
8. **Pretest vs posttest:** pretest 0 kez çalışabilir, posttest en az 1 kez çalışır.
9. **Ada loop variable scope’u** sadece loop’tur; Fortran 90 loop sonrası undefined yapar; C/Java son değer bırakır.
10. **Java break label** = etiketli break; iç içe loop’tan dış loop’a çıkar. C’nin break’i sadece tek loop’tan çıkar.
11. **Ruby block** bir method’a parametre olarak gider; `yield` ile çağrılır. Java/C# foreach standart `Iterable/IEnumerator` ile çalışır.
12. **Generator (Python yield):** method bir thread gibi davranır; her çağrıda kaldığı yerden devam eder.
13. **goto eliminasyonu eksiksiz değildir:** Java goto’yu yasakladı ama labeled break/continue ile dolaylı goto sağlar.
14. **Guarded command nondeterminism:** birden fazla guard true ise herhangi biri seçilir; programın doğruluğu seçime bağlı olmamalı.
15. **Tüm guard’lar false** ise guarded command **run-time error** verir; “hiçbir şey yapmama” davranışı yoktur. Bu programcayı tüm durumları listelemeye zorlar.
16. **Scheme COND:** ilk #T dönen predicate’in expression’ı sonuçtur; ELSE olmazsa ve hiçbiri true değilse sonuç tanımsızdır.

9 Review Questions — Soru Çözümleri

S1. Control structure’un tanımı nedir?

Cevap

Control structure, bir control statement ile bu statement’ın yürütmesini kontrol ettiği statement koleksiyonunun bütününe verilen isimdir. Yani sadece `if`, `while` gibi anahtar statement değil; o statement’ın yönettiği gövde (body) ile birlikte ele alınan yapıdır.

S2. Böhm ve Jacopini flowchart'lar hakkında ne kanıtladılar?**Cevap**

1966'da Böhm ve Jacopini, *flowchart ile ifade edilebilen tüm algoritmaların* yalnızca iki control statement ile (iki yollu seçim ve mantıksal kontrollü iteration) kodlanabileceğini kanıtladılar. Bu sonuç **unconditional branch** (goto) statement'ının teorik olarak gereksiz ("superfluous") olduğunu gösterir.

S3. Block'un tanımı nedir?**Cevap**

Block, bir compound statement'tır; aynı zamanda kendi içinde **yerel değişken tanımlamalarına** (local declarations) izin veren bir sözdizimsel birimdir. Block, ayrı bir scope oluşturur. C-tabanlı dillerde { ... } ile sınırlandırılır; Pascal'da begin ... end ile.

S4. Tüm selection ve iteration control statement'lar için ortak tasarım sorusu nedir?**Cevap**

Tek ortak tasarım sorusu: **control structure çoklu giriş noktasına sahip olmalı mı?** Yani body, ilk statement dışında başka bir noktadan da girilebilmeli mi? Modern görüş: hayır; multiple entries esneklikten çok karmaşıklık katar. Multiple entries zaten yalnızca goto ve statement label içeren dillerde mümkündür.

S5. Selection structure'ları için tasarım soruları nelerdir?**Cevap**

Selection (özellikle two-way) için üç temel tasarım sorusu:

- Seçimi kontrol eden expression'ın **form**'u ve **tip**'i ne?
- **then** ve **else** clause'ları nasıl belirtilir? (tek statement, compound, statement sequence)
- **Nested selector**'ların anlamı nasıl belirlenir? (dangling else)

S6. Python'un compound statement tasarımının olağandışı yanı nedir?**Cevap**

Python compound statement'ı belirtmek için **küme parantezi ({} ya da begin/end gibi bir reserved word kullanmaz**; bunun yerine **indentation** (girinti) kullanır. Aynı seviyede girintilenmiş statement'lar aynı bloğa aittir. Bu nedenle Python'da girinti, kozmetik bir tercih değil **semantik** bir araçtır.

S7. F#'ta selector hangi durumlarda else clause'una sahip olmak zorundadır?**Cevap**

F# selector'ı bir **expression**'dır ve değer döndürür. Eğer then clause'u **normal bir tip** (int, string gibi) değer döndürüyorsa, else clause'u **zorunludur** ve aynı tipte değer döndürmelidir. Yalnızca then clause'u unit tipinde (), "değer yok" ise else olmadan kullanılabilir.

S8. Two-way selector’larda nesting problemine yaygın çözümler nelerdir?**Cevap**

- **Static semantic kuralı (Java, C, C++, C#):** else daima en yakın eşleşmemiş then’le eşleşir; alternatif anlam için iç `if` compound’a alınır.
- **Zorunlu compound (Perl):** tüm then/else clause’ları küme parantezleri ile blok olur; ambiguity doğamaz.
- **Terminator reserved word (Ruby, Ada):** `end` / `end if` ile selection’ın bitiş noktası açıkça belirtilir.
- **Indentation (Python, F#):** girinti seviyesi semantik anlam taşır.
- **else-siz if’i yasaklama (ML):** dangling else doğal olarak doğamaz.

S9. Multiple-selection statement’lar için tasarım soruları nelerdir?**Cevap**

1. Seçimi kontrol eden expression’ın form’u ve tip’i ne?
2. Seçilebilir segment’ler nasıl ifade edilir? (tek statement, compound, sequence)
3. Tek bir yürütmede yalnız bir segment mi yürütülür, yoksa birden fazlasına akış mümkün mü? (fall-through)
4. Case değerleri nasıl belirtilir? (sabitler, range, Boolean expr)
5. Selector expression hiçbir case’e uymazsa ne olur?

S10. Bir multiple selection statement’ında birden fazla segment’in yürütülmesine izin verme kararı hangi iki dil özelliği arasında bir trade-off’tur?**Cevap**

Reliability vs flexibility arasında bir trade-off’tur. Birden fazla segment’e izin vermek (C’deki fall-through) kasıtlı kullanım için **flexibility** kazandırır; ama `break` unutulursa sessizce mantığı bozduğundan **reliability**’yi düşürür. C# bu trade-off’ta reliability tarafına geçmiştir.

S11. C’nin multiple-selection statement’ının olağandışı yanı nedir?**Cevap**

C’nin `switch`’inde **implicit branch** yoktur: bir case segmenti tamamlanınca otomatik olarak `switch`’ten çıkmaz, kontrol bir sonraki case’e **akar** (fall-through). Çıkmak için açıkça `break` yazmak gerekir. Ayrıca case etiketleri sözdizimsel olarak neredeyse normal etiketler gibi muamele görür; bu da kötüye kullanım için kapı bırakır.

S12. C’nin switch’i hangi önceki dile dayanır?

Cevap

ALGOL 68'in çoklu seçim statement'ına dayanır. ALGOL 68 da implicit branch içermezdi; C bu davranışı miras almıştır.

S13. C#'ın switch'i C'ninkine kıyasla nasıl daha güvenlidir?**Cevap**

C#, bir static semantic kuralla **implicit fall-through'u yasaklar**: her selectable segment ya `break` ile ya `goto case/goto default` ile ya da `return/throw` gibi başka bir koşulsuz dal ile sonlanmak zorundadır. Bu, `break` unutmaktan kaynaklı sessiz hataları **derleme zamanında** yakalar; reliability'yi belirgin biçimde artırır. Ayrıca C#'ta case değerleri ve kontrol string olabilir.

S14. Tüm iterative control statement'lar için tasarım soruları nelerdir?**Cevap**

1. Iteration nasıl kontrol edilir? (logical / counting / kombine)
2. Kontrol mekanizması loop'un neresinde olur? (top / bottom / user- located)

S15. Counter-controlled loop statement'lar için tasarım soruları nelerdir?**Cevap**

1. Loop variable'ın ve loop parameters'ın **tip'i** ve **scope'u** ne?
2. Loop body içinde loop variable / parameters **değiştirilebilir mi**? Değiştirme loop akışını etkiler mi?
3. Loop parameters **her iterasyonda mı yoksa yalnız bir kez mi** değerlendirilir?
4. Loop sona erdikten sonra loop variable'ın değeri nedir?

S16. Pretest loop nedir? Posttest loop nedir?**Cevap**

Pretest loop: kontrol koşulu loop body'sinden *önce* değerlendirilir. Koşul başlangıçta false ise body **hiç** çalışmayabilir. Örnek: C `while`.

Posttest loop: kontrol koşulu body'den *sonra* değerlendirilir. Body **her zaman en az bir kez** çalışır. Örnek: C `do-while`.

S17. C++'ın for statement'ı ile Java'nınki arasındaki fark nedir?**Cevap**

Java'da `for`'un kontrol expression'ı **yalnızca** boolean tip olabilir; C++'ta hem aritmetik hem Boolean expression kullanılabilir. Ayrıca Java'da `goto` olmadığı için `for` body'sine dışarıdan dallanma yapılamaz; C/C++ buna izin verir. Genel form (init, kontrol, güncelleme) iki dilde de aynıdır ve init kısmında declaration yapılabilir.

S18. C'nin for'u diğer pek çok dile kıyasla neden daha esnek?

Cevap

Üç ayrı expression yuvasına izin verir; her biri **birden fazla expression**'dan (virgül operatörüyle) oluşabilir. Bu sayede:

- Birden çok loop variable kullanılabilir.
- Loop variable herhangi bir tip olabilir (integer, float, pointer).
- Init veya update kısmı tamamen boş bırakılabilir; kontrol boş olursa sonsuz döngü.
- `for` aslında **counting da, logical da** olabilir.
- Tüm değişkenler body içinde değiştirilebilir.

Bu esneklik writability'yi artırır; ama yanlış kullanım reliability'yi düşürebilir.

S19. Python'da range fonksiyonu ne yapar?**Cevap**

`range`, sayı dizisi üretir; 1, 2 veya 3 parametre alır:

- $\text{range}(n) \rightarrow 0, 1, \dots, n-1$
- $\text{range}(a, b) \rightarrow a, a+1, \dots, b-1$
- $\text{range}(a, b, s) \rightarrow a, a+s, a+2s, \dots$ (b'ye ulaşmadan)

Üst sınırı asla içermez. Counter-controlled loop'larda `for x in range(...)` kalıbı kullanılır.

S20. Hangi çağdaş diller goto içermez?**Cevap**

Java, Python ve Ruby `goto` içermez. Bunun yerine etiketli/etiketsiz `break`, `continue`, `return` gibi disiplinli dallanma araçları sunarlar.

S21. Logically controlled loop statement'ları için tasarım soruları nelerdir?**Cevap**

1. Kontrol **pretest** mi **posttest** mi olmalı?
2. Logically controlled loop, counter-controlled loop'un özel formu mu olmalı, yoksa **ayrı statement** mi olmalı?

S22. User-located loop control statement'lar neden icat edildi?**Cevap**

Bazı durumlarda kontrolün loop'un başında veya sonunda olması doğal değildir; çıkış **ortada** oluşmalıdır (örneğin başarılı bir arama, beklenmedik durum, hata). Bu tür durumlarda `goto` kullanmak okunabilirlik açısından maliyetlidir; `break / last` gibi kısıtlı dallanma mekanizmaları, yaygın bir `goto` ihtiyacını disiplinli ve okunabilir biçimde karşılar.

S23. User-located loop control mekanizmaları için tasarım soruları nelerdir?**Cevap**

1. Koşullu mekanizma exit'in **kendi parçası** mı, yoksa exit ayrıca koşulsuz mu olmalı?
2. Yalnız **tek bir loop body**'sinden mi çıkılır, yoksa **çevreleyen loop'lar**'dan da çıkılabilir mi?

S24. Java'nın break statement'ının C'nin break'ine göre ne avantajı vardır?**Cevap**

Java'da break **etiketli** olabilir: `break outerLoop;` söz dizimi, iç içe loop'ların **herhangi bir dış katmanından** doğrudan çıkmaya izin verir. C'nin break'i yalnızca **en içteki** switch veya loop'tan çıkar; iç içe loop'tan dış loop'a çıkmak için ek bir flag değişkeni veya goto gerekir. Java'nın tasarımı bu yaygın paterni daha okunabilir ve daha güvenli yapar.

S25. C++'ın break statement'ı ile Java'nınki arasındaki farklar nelerdir?**Cevap**

- C++'ta break **etiket alamaz**; en içteki switch veya loop'tan çıkar.
- Java'da break **etiketli** olabilir; `break label;` ile dış loop'tan çıkılabilir.
- Java'da break `labeled statement` dışına atlatabilir (sadece loop değil).
- C++'ta dış loop'tan çıkmak için `goto` veya bayrak değişkeni gerekir; Java'da bu temiz biçimde yapılır.

S26. User-defined iteration control nedir?**Cevap**

User-defined iteration control, kullanıcı tanımlı bir veri yapısının elemanlarını dolaşan, yine kullanıcı tanımlı bir **iterator** fonksiyonu çağıran iteration statement'ıdır. Iterator her çağrıda bir sonraki elemanı verir, sonuncuda bittiğini bildirir; yani **history-sensitive** bir fonksiyondur. Özellikle abstract data type ve OOP'de yaygındır; C foreach, Java foreach, C# IEnumerator, Python generator, Ruby block bu kategoriye örneklerdir.

S27. Hangi Scheme fonksiyonu multiple selection statement'ı implemente eder?**Cevap**

COND özel formu (special form). Birden fazla (`predicate expression`) çiftini sırayla değerlendirir; ilk `#T` dönen predicate'in `expression`'ı sonuçtur. İsteğe bağlı bir `ELSE` clause'u ile sonlandırılabilir.

S28. Functional dil repetition'ı nasıl implemente eder?**Cevap**

Saf functional dillerde değişken (ve dolayısıyla counter) yoktur; bu yüzden loop yerine **recursion** kullanılır. Bir fonksiyon kendisini çağırarak tekrarlamayı sağlar; sonlanma, recursion'ın temel

adımına ulaşıldığında olur. Compiler'lar tipik olarak **tail call optimization** ile bu rekürsif çağrılar stack büyütmeden iteration'a dönüştürür.

S29. Ruby'de iterator'lar nasıl implemente edilir?

Cevap

Ruby iterator'ları, **block** alan özel method'lardır. Bir block (`{ ... }` veya `do ...end`), method'a anonim bir parametre olarak geçer. Method, `yield` statement'ı ile bu block'u çağırır; her çağrıda block'un kodu (varsa parametreyle) çalışır. Önceden tanımlı iterator'lar (`each`, `times`, `upto`, `map`) yaygın veri yapıları için bulunur; programcı kendi iterator'larını yazabilir.

S30. Hangi yaygın dil, kendi önceden tanımlı veri yapıları üzerinde açıkça çağrılabilen önceden tanımlı iterator'lar tanımlar?

Cevap

Ruby. Diziler, hash'ler, tamsayılar gibi nesneler üzerinde `each`, `times`, `upto`, `downto`, `map`, `select` gibi iterator method'ları doğrudan çağrılabilir. Diğer modern diller (Java, C#, Python) genelde implicit iterator (`foreach`) kullanır; Ruby'de iterator'lar explicit bir method çağrısıdır.

S31. Hangi yaygın programlama dili, tasarımının bir kısmını Dijkstra'nın guarded command'larından ödünç alır?

Cevap

Haskell. Fonksiyon tanımlarındaki **guards** (`| condition = result`) sözdizimi doğrudan Dijkstra'nın guarded command'larından esinlenmiştir. Ayrıca **CSP** (Hoare, 1978) modelinde ve dolayısıyla concurrent dillerde (Ada `select`, `occam`) guarded command semantiğinin izleri açıkça görülür.

10 Problem Set — Soru Çözümleri

P1. Birleşik counting + logical loop'a ihtiyaç duyulan üç durumu açıklayın.

Cevap

1. **Sınırlı arama:** bir dizide bir değeri ararken, hem *en fazla N eleman* sınırı hem de *bulunduğunda dur* koşulu vardır:

```
for (i = 0; i < N && !found; i++) { ... }
```

2. **Iterative numerical methods:** Newton-Raphson gibi yakınsama algoritmalarında *maksimum iterasyon* (counter, sonsuz döngüden korunma) ile *tolerans altına düşünce dur* (logical) birlikte uygulanır.

3. **Buffer doldurma:** bir akıştan en fazla N bayt oku, ama akış erken biterse de dur:

```
for (i = 0; i < N && (c = getchar()) != EOF; i++) buf[i] = c;
```

Bu durumlarda C `for`'u uygundur; Ada gibi dillerde counter loop içinde `exit when` ile sağlanır.

P2. CLU'nun iterator özelliğinin avantaj ve dezavantajlarını analiz edin.**Cevap**

CLU (Liskov, 1977) iterator'ları, `yield` mekanizmasıyla bir veri yapısı üzerinde elem üreten ayrı sözdizimsel birimlerdir.

Avantajlar:

- Veri yapısının iç temsili (representation) gizlenir; kullanıcı iterator interface'yle çalışır (data abstraction).
- History-sensitivity sözdizimsel olarak desteklenir; programcı state taşımak zorunda değildir.
- Loop ve iterator gevşek bağlıdır; aynı veri yapısı üzerinde birden çok iterator (sıralı, ters, key-only) tanımlanabilir.

Dezavantajlar:

- Implementation karmaşıklığı yüksektir; coroutine benzeri yığın yönetimi gerektirir, performans maliyeti vardır.
- Her iterator çağrısı method/coroutine geçişi getirir; sıkı sayısal döngülerde overhead.
- Dil sözdizimine yeni kavram ekler; öğrenme eğrisi.

Ruby `block` ve Python `yield generator` bu fikrin daha modern, daha yaygın varisleridir.

P3. Ada control statement setini C# ile karşılaştırın; hangisi daha iyi, neden?**Cevap**

Özellik	Ada	C#
Selection	<code>if .. elsif .. end if,</code> <code>case</code> (no fall-through)	<code>if/else if/else,</code> <code>switch</code> (no implicit fall-through)
Loop var. scope	sadece loop	son değer (<code>int</code> tanımlanmışsa loop scope)
goto	var, kısıtlı	var, switch ile entegre
Loop exit	<code>exit when condition;</code>	<code>break</code>
Multi-exit	label ile	label yok, goto var

Değerlendirme: Ada **reliability** ve **readability** açısından önde: loop variable scope'u sıkıdır, case fall-through yoktur, `exit when` doğal okunur. C# **writability** ve modern ekosistem (foreach, generic, LINQ) açısından önde. Hangisinin "daha iyi" olduğu domain'e bağlıdır: yüksek güvenlikli sistemler için Ada, genel purpose uygulama için C#.

P4. Compound statement'larda farklı kapanış reserved word'leri kullanmanın artıları/eksileri nelerdir?**Cevap****Artılar:**

- **Readability:** `end if`, `end loop` gibi ifadeler hangi yapının bittiğini açıkça belirtir; iç içe yapılarda `{ }` yığını saymaktan iyidir.

- Dangling else gibi belirsizliği ortadan kaldırır.
- Compiler hata mesajları daha hedefli olur.

Eksiler:

- Daha fazla yazma (verbosity); writability açısından dezavantaj.
- Tipo riski: `end if` yerine `end loop` yazma hatası derleme sırasında yakalansa da kafa karıştırır.
- Fonksiyonel sözdizimi (one-liner) daha zor olur.

Ada, Ruby, ALGOL 68 (`fi`, `od`) bu yaklaşımı benimser; C ailesi sadeliği seçer.

P5. Python'un compound statement için indentation kullanmasının artıları/eksileri nelerdir?**Cevap****Artılar:**

- **Görsel yapı = mantıksal yapı:** Python'da gördüğümüz girinti doğrudan bloğu temsil eder; yanıltıcı girintilenmiş kod (C/Java'daki gibi) imkânsızdır.
- Kod tabanları arasında stil tutarlılığı doğal olarak sağlanır.
- Daha az noktalama işareti, daha az gürültü.

Eksiler:

- Tab vs space karışıklığı yıkıcı olabilir; hatalar görünmez kalır.
- Otomatik kod oluşturma araçları için zorlu; girintiyi koruyan taşıma/yapıştırma gerekir.
- Bir bloğu başka bir scope'a taşıırken tüm satırların yeniden girintilenmesi gerekir.
- Tek satırlık compound mümkün değil (`if x: do(); end` gibi).

P6. ALGOL 68'in case-esac (kapanış reserved word'ü baştakinin tersi) tasarımının okunabilirlik problemleri.**Cevap**

ALGOL 68'in `if-fi`, `case-esac`, `do-od` tasarımı zariftir, ama:

- **Tipo hatası riski:** `esac` ve `case` arasında yer değişikliği yazılırken karakter atlama veya transpoze oluşabilir (`esca`, `caes`); compiler genelde yakalar ama hata mesajı konum açısından kafa karıştırıcı olabilir.
- Doğal dilde olmayan kelimeler okuma akışını yavaşlatır (`fi`, `od`).
- İç içe yapılarda hangi `esac`'ın hangi `case`'e ait olduğu uzun bloklarda kayıp olur (`end case` kadar açık değil).
- Klavyede `f-i` ve `i-f` sıkça karıştırılır.

Modern diller (Ruby, Ada) bu yerine `end / end if` gibi açık ifadeleri tercih eder.

P7. Knuth (1974) makalesini özetleyen ve goto tartışmasının iki tarafını sunan kısa bir özet.**Cevap**

Knuth'un "*Structured Programming with go to Statements*" (1974) makalesi, Dijkstra (1968)'in tam yasaklama görüşüne karşı dengeli bir duruş sergiler.

Knuth'un argümanları (goto lehine, kısıtlı):

- Bazı algoritmalarda goto kullanmak hem performans hem readability açısından alternatif (boolean flag, ekstra değişken) üstündür.
- Erken çıkış, hata yönetimi, ortak çıkış noktası gibi paternler goto ile doğal yazılır.
- "Premature optimization" kötüdür ama ölçülmüş darboğaz için goto haklı olabilir.

Dijkstra tarafı:

- Goto static program metni ile dynamic execution arasındaki ilişkiyi bozar; correctness ispatı zorlaşır.
- Spagetti kod, bakım kabusudur.

Sentez: Bugün modern dillerin çoğu goto'yu kaldırmış ya da çok sınırlamıştır; Knuth'un kullanım durumlarının çoğu `break`, `continue`, `return` ile karşılanmıştır.

P8. Knuth'un birden çok exit'e izin veren loop control statement'ının operational semantics description'ı.**Cevap**

Knuth, body içinde `exit if condition` formunda birden çok çıkış sunan bir loop önerir:

```
loop
  S1
  exit if E1
  S2
  exit if E2
  ...
  Sn
end loop
```

Operational semantics:

```
top:
  [code for S1]
  if E1 then goto out
  [code for S2]
  if E2 then goto out
  ...
  [code for Sn]
  goto top
out:
```

Bu yapı, "ortadan çıkış" (exit-in-the-middle) paternini doğal ifade eder ve bayrak değişkeni gerektirmez; Ada'nın `loop ...exit when ...end loop` yapısı bu fikrin doğrudan inkarnasyonudur.

P9. Java’da kontrol statement’larında yalnız Boolean expression’a izin vermenin (C++’taki gibi aritmetik de değil) artı/eksileri.

Cevap

Artılar:

- **Reliability:** `if (x = 5)` gibi tipik atama-yerine- karşılaştırma hataları derleme zamanında yakalanır.
- Type system tutarlılığı: koşul yerinde sadece anlamı net olan Boolean kabul edilir; “sıfır = false” kuralının dolaylı tip dönüşümünden kurtulur.
- Okuyucunun zihinsel yükü azalır: `while (x)` ne demek? – Java’da “`x.boolean` değeri `true` iken” açıkken; C’de “`x` sıfır değil iken” gibi düşünmek gerekir.

Eksiler:

- Idiom kaybı: C/C++ programcısının kullandığı `while (n-)` gibi pratik kalıplar yazılamaz; `while (n- > 0)` gibi açık yazılması gerekir.
- Pointer/null kontrolünde `if (ptr)` kısaltması yok; `if (ptr != null)` açıkça yazılır (writability açısından küçük kayıp, readability lehine).

Genel kabul: Java’nın seçimi reliability lehine doğru bir trade-off’tur.

P10. Python `for` statement’ında `else` clause’unun işe yarayacağı bir programlama durumu.

Cevap

Python’da `for-else` yapısının `else` kısmı, döngü **break** ile değil normal sona ererse çalışır. En tipik kullanım **arama**’dır:

```
for item in collection:
    if predicate(item):
        process(item)
        break
else:
    # only runs if loop completed without break:
    handle_not_found()
```

Bu yapı `found` bayrağı tutmaktan daha okunaklıdır. Diğer kullanımlar: tüm elemanların bir koşulu sağlayıp sağlamadığını doğrulama, validation döngüsü.

P11. Posttest loop gerektiren üç spesifik durum.

Cevap

1. **Kullanıcıdan input alma:** en az bir kez sormak gerekir, geçersiz girişte tekrar sor.

```
do {
    printf("Enter (1-9): ");
    scanf("%d", &x);
} while (x < 1 || x > 9);
```

2. **Sayının basamak sayısını bulma:** 0 olsa bile bir basamağı vardır.

```
do {
    digits++;
    n /= 10;
} while (n > 0);
```

3. **Ağ retry mantığı:** en az bir deneme yap, başarısız olursa belirli sınıra kadar tekrar dene.

```
do {
    success = try_request();
    attempts++;
} while (!success && attempts < MAX_ATTEMPTS);
```

Ortak nokta: “en az bir kez yapma” garantisi gerekiyor.

P12. C loop statement’ına kontrol transferine neden izin verilebilir? (Kontrolün loop’un içine atlanması)

Cevap

C, goto ve switch case etiketleri sayesinde loop body’nin ortasına dallanmaya izin verir. Olası sebepler:

- **Düşük seviyeli optimizasyon:** elle yazılmış makine-yakını kod paternlerine (Duff’s Device gibi) izin verir; switch ve do-while’ı birleştiren ünlü loop unrolling tekniği bu sayede yazılır.
- **Tarihsel donanım uyumu:** C’nin hedefi makine talimatlarına yakın kontroldür; HW destekli loop talimatlarını yansıtır.
- **Tek girişin compiler-enforced olmaması:** C dilin felsefesi “programcıya güven”dir; restriction koymak istemez.

Maliyet: okunabilirlik felakete dönüşebilir; bu nedenle Java goto’yu kaldırmıştır. Modern coding standards (MISRA-C) bu özelliğin kullanımını yasaklar.

11 Programming Exercises — Ek Çözümler

PE1. Verilen pseudocode’u C/C++/Java/C#, Python ve Ruby’de loop yapısı kullanarak yeniden yazın. Tüm değişkenler integer.

Cevap

Pseudocode:

```
k = (j + 13) / 27
loop:
    if k > 10 then goto out
    k = k + 1
    i = 3 * k - 1
    goto loop
out: ...
```

(a) C/C++/Java/C# (while):

```

int k = (j + 13) / 27;
while (k <= 10) {
    k = k + 1;
    i = 3 * k - 1;
}

```

(b) Python:

```

k = (j + 13) // 27
while k <= 10:
    k = k + 1
    i = 3 * k - 1

```

(c) Ruby:

```

k = (j + 13) / 27
while k <= 10
    k = k + 1
    i = 3 * k - 1
end

```

Readability/Writability: Üç sürüm de işlevsel olarak aynı, ama Ruby `end` ile en açık blok sınırını verir; Python en az noktalama ile en sade kod olur (*writability*); C-tabanlı kod `{ }` ile yapısı net (*readability*). Genel olarak Python her iki ölçütte de iyi denge sağlar.

PE2. PE1’i floating-point ile yeniden yapın; $k = k + 1$ yerine $k = k + 1.2$.

Cevap**(a) C/C++/Java/C#:**

```

double k = (j + 13.0) / 27.0;
while (k <= 10.0) {
    k = k + 1.2;
    i = 3.0 * k - 1.0;  /* i is now double */
}

```

(b) Python: Python tip dinamik olduğu için kod neredeyse aynıdır:

```

k = (j + 13.0) / 27.0
while k <= 10.0:
    k = k + 1.2
    i = 3.0 * k - 1.0

```

(c) Ruby: Ruby’de de tip dinamik:

```

k = (j + 13.0) / 27.0
while k <= 10.0
    k = k + 1.2
    i = 3.0 * k - 1.0
end

```

Not: Floating-point eşitlik karşılaştırmasından kaçınılmalı; `<=` kullanmak burada güvenli. Static-typed dillerde `i` değişkeninin tipini ön açıkça belirtmek (`double`) önemli; dinamiklerde ekstra çaba yok.

PE3. Çoklu if’i multiple-selection statement ile yeniden yazın.

Cevap

Orijinal:

```

if ((k == 1) || (k == 2)) j = 2 * k - 1;
if ((k == 3) || (k == 5)) j = 3 * k + 1;
if (k == 4) j = 4 * k - 1;
if ((k == 6) || (k == 7) || (k == 8)) j = k - 2;

```

(a) C/C++/Java/C# (switch):

```

switch (k) {
    case 1: case 2:
        j = 2 * k - 1; break;
    case 3: case 5:
        j = 3 * k + 1; break;
    case 4:
        j = 4 * k - 1; break;
    case 6: case 7: case 8:
        j = k - 2; break;
}

```

(b) Python (elif):

```

if k in (1, 2):
    j = 2 * k - 1
elif k in (3, 5):
    j = 3 * k + 1
elif k == 4:
    j = 4 * k - 1
elif k in (6, 7, 8):
    j = k - 2

```

(c) Ruby (case):

```

case k
when 1, 2      then j = 2 * k - 1
when 3, 5      then j = 3 * k + 1
when 4         then j = 4 * k - 1
when 6, 7, 8   then j = k - 2
end

```

Değerlendirme: Ruby case burada en kompakt ve okunabilir olanıdır; when 1, 2 doğrudan grupları ifade eder. Python in (...) kalıbı readable. C switch biraz daha verbose (her grup için ayrı satır) ama tüm dillerde olduğu için en taşınabilir olanıdır.

PE4. Aşağıdaki C kodunu goto ve break kullanmadan yeniden yazın.

Cevap

Orijinal:

```

j = -3;
for (i = 0; i < 3; i++) {
    switch (j + 2) {
        case 3:
        case 2: j--; break;
        case 0: j += 2; break;
    }
}

```

```

        default: j = 0;
    }
    if (j > 0) break;
    j = 3 - i;
}

```

Yeniden yazım (goto/break yok, bayrak değişkeni ile):

```

j = -3;
int stop = 0;
for (i = 0; i < 3 && !stop; i++) {
    int s = j + 2;
    if (s == 3 || s == 2) {
        j--;
    } else if (s == 0) {
        j += 2;
    } else {
        j = 0;
    }
    if (j > 0)
        stop = 1;
    else
        j = 3 - i;
}

```

Değerlendirme: stop bayrağı break'i yerleştirir; switch if/else if zincirine dönüşür. Readability açısından orijinal versiyon (kontrollü break ile) daha temiz. Bu örnek break'in disiplinli kullanımının neden tercih edildiğini iyi gösterir.

PE5. Rubin'in (1987) all-zero satır arama kodunu goto kullanmadan C/C++/Java/C#'ta yeniden yazın.

Cevap

Orijinal (goto'lu):

```

for (i = 1; i <= n; i++) {
    for (j = 1; j <= n; j++)
        if (x[i][j] != 0)
            goto reject;
    println("First all-zero row is:", i);
    break;
reject: ;
}

```

Java versiyonu (etiketli break ile, goto'suz):

```

outer:
for (int i = 1; i <= n; i++) {
    for (int j = 1; j <= n; j++) {
        if (x[i][j] != 0)
            continue outer;
    }
    System.out.println("First all-zero row is: " + i);
    break;
}

```

C/C++ (bayrak değişkeniyle):

```

int found = 0;
for (i = 1; i <= n && !found; i++) {
    int allzero = 1;
    for (j = 1; j <= n; j++) {
        if (x[i][j] != 0) { allzero = 0; break; }
    }
    if (allzero) {
        printf("First all-zero row is: %d\n", i);
        found = 1;
    }
}

```

Karşılaştırma: Java versiyonu (continue outer) en temiz sayılır — niyet açıkça okunur. C bayrak versiyonu daha gürültülü ama taşınabilir. Rubin’in noktası, bazı paternlerde goto’nun gerçekten daha açık olduğuydu; Java’nın etiketli continue/break’i bu paterni goto olmadan karşılar.

PE6. Üç değişkenin (first, second, third) max, mid, min’e atanması; nested ile ve nested olmadan iki çözüm.

Cevap**Çözüm 1 — Nested if:**

```

if (first >= second) {
    if (first >= third) {
        max = first;
        if (second >= third) { mid = second; min = third; }
        else { mid = third; min = second; }
    } else {
        max = third; mid = first; min = second;
    }
} else {
    if (second >= third) {
        max = second;
        if (first >= third) { mid = first; min = third; }
        else { mid = third; min = first; }
    } else {
        max = third; mid = second; min = first;
    }
}

```

Çözüm 2 — Düz seçim (nested olmadan):

```

max = first;
if (second > max) max = second;
if (third > max) max = third;

min = first;
if (second < min) min = second;
if (third < min) min = third;

mid = first + second + third - max - min;

```

Karşılaştırma:

Ölçüt	Nested	Düz
Karmaşıklık	6 yol; takip etmek zor	3 + 3 sıralı kontrol; sade
Reliability	Bir dal eksik kalsa hata kalır	Her dal bağımsız test
Performans	Erken dallanma; az karşılaştırma	Tüm karşılaştırmalar yapılıp (eşit)
Reading effort	Yüksek	Düşük

Düz versiyon hem daha güvenilir hem de daha okunaklıdır. Toplam-eksiltme hilesi ($mid = a+b+c-\max-\min$) elegant ama integer overflow riskini taşır; production'da mid 'i ayrı seçim ile bulmak güvenlidir.

PE7. PE4'teki kodu C'de `if` ve `goto` ile yeniden yazın.

Cevap

```

j = -3;
i = 0;
loop:
    if (i >= 3) goto end;

    if (j + 2 == 3 || j + 2 == 2) {
        j--;
        goto after_switch;
    }
    if (j + 2 == 0) {
        j += 2;
        goto after_switch;
    }
    j = 0;

after_switch:
    if (j > 0) goto end;
    j = 3 - i;
    i++;
    goto loop;
end:

```

Değerlendirme: Çalışır ama okunabilirliği kötü. Modern bir kod incelemesinde reddedilirdi. Tek faydası, eski makine kodu okumayı öğretmek. Bu örnek `goto`'nun teorik gücünün pratikte neden kayba uğradığını gösteriyor.

PE8. PE4'teki kodu Java'da `switch` kullanmadan yeniden yazın.

Cevap

```

int j = -3;
outer:
for (int i = 0; i < 3; i++) {
    int s = j + 2;
    if (s == 3 || s == 2) {
        j--;
    } else if (s == 0) {

```

```

        j += 2;
    } else {
        j = 0;
    }
    if (j > 0)
        break outer;
    j = 3 - i;
}

```

Değerlendirme: Java’da switch olmadan if/else if zinciri kullanılır. Etiketli break outer sayesinde dış loop’tan çıkış temizdir. Bu versiyon orijinaline göre belki bir miktar daha verbose ama goto/break-from-switch ihtiyacı yoktur.

PE9. Aşağıdaki Scheme COND çağrısını C’ye çevirin ve sonucu y’ye atayın.

Cevap

Orijinal:

```

(COND
  ((> x 10) x)
  ((< x 5) (* 2 x))
  ((= x 7) (+ x 10))
)

```

Not: Scheme COND predicate’leri **sırayla** değerlendirir. $x > 10$ önce kontrol edildiği için, $x = 7$ veya $x < 5$ branch’leri ancak $x \leq 10$ ise sıraya gelir. $x = 7$ koşulu, ne > 10 ne < 5 olduğu için ulaşılabilir. **C versiyonu:**

```

int y;
if (x > 10) {
    y = x;
} else if (x < 5) {
    y = 2 * x;
} else if (x == 7) {
    y = x + 10;
} else {
    /* COND has no ELSE: result undefined */
    /* defensive default: leave y unchanged
       or set sentinel value */
    y = -1;
}

```

Önemli ayrıntı: Scheme COND’da hiçbir predicate true değilse ve ELSE yoksa sonuç *tanımsızdır*. C’ye birebir aktarımda bunun bir karşılığı olmadığı için programcı sentinel/default seçmek zorundadır. Bu, dil semantiğindeki ince farkın çevirideki tipik bir zorluğunu gösterir.